

[Home](#) » [Posts](#)

From PageRank to word2vec to Attention

October 8, 2026 · 31 min · 6483 words · Varun Tulsian

► [Table of Contents](#)

You are defined by the company you keep: one saying, three algorithms.

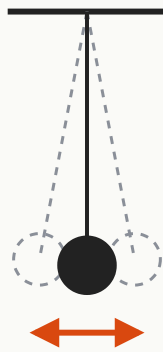
Is minimax even relevant?

I was explaining to Tanmay how PageRank works: in-links, out-links, and a loop that runs until the scores stop changing. Somewhere in that conversation he made a point. In college we learn basic things, like the minimax algorithm. He felt they are not really relevant anymore.

I disagreed. You need to understand the foundations to build your intuition. When you have that intuition, you can apply it.

At Fermi, my colleague Madhu said in one of his physics lessons that simple harmonic motion is the foundation of physics. To me, that is the same intuition.

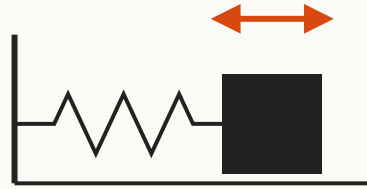
pendulum



simple harmonic motion

$$F = -kx$$

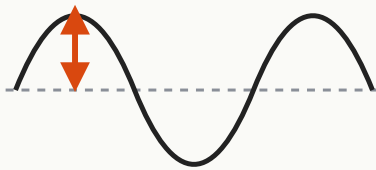
spring



simple harmonic motion

$$F = -kx$$

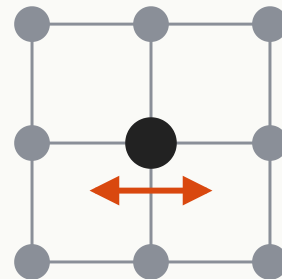
sound wave



simple harmonic motion

$$F = -kx$$

atom in a crystal



simple harmonic motion

$$F = -kx$$

Four systems, one equation.

THE PHILOSOPHY

Every powerful algorithm started as an old intuition about the world, made precise. The way it happens has three steps. First, someone makes an argument about the world, almost a philosophical one. Then they write the argument down as maths. Then they solve the maths, and an algorithm falls out.

If you only learn the algorithm, you can use it, or change it a little. If you learn the argument behind it, the algorithm becomes a building block. You can take it apart, swap a piece, and build something new. It feels like LEGO: you build a block, then you complicate it.

This post follows one argument through three famous algorithms: PageRank, word2vec and attention. The goal is that by the end you see one idea where you used to see three, and you can apply it to a problem of your own.

The saying

THE PHILOSOPHY

People say: you are defined by the company you keep.

Take it literally for a moment. It does not say what you are made of. It says you are what you are next to. To learn about you, look at your friends.

That is a claim about similarity. Two people who keep the same company should be similar, even if they have never met. A thing with no company tells us nothing.

We can draw the saying as a table. Take tea, coffee and a carburettor, and put them in the rows. Put the words they often appear next to in the columns: cup, drank, morning, engine. Tea and coffee get high counts under cup, drank and morning. The carburettor gets its count under engine. So the rows are things, the columns are their company, and each cell says how strongly a thing keeps that company. Everything in this post starts from a table like this. The only questions are what the rows and columns are, and what we do with the table once we have it.

	their company			
	cup	drank	morning	engine
tea				
coffee				
carburettor				

bigger dot = they appear together more often

The saying, drawn as a table: things in the rows, their company in the columns.

So here is the question for the rest of this post. What happens when you take this one sentence literally, for web pages, for words and for people?

Pages: pass it around until it settles

The problem in 1998

Start with the web in the late 1990s. Search engines ranked a page mostly by its own text: how well the words on the page matched the words in the query. That had a flaw. The author of a page controls its text, so the author can make it match anything. Anything that counted words on the page was easy to manipulate¹, and very short pages that held little more than the query often ranked first².

The web was also growing fast, while people still read only the first few dozen results². So the problem was not finding pages that match. It was putting the right ten pages on top.

Brin and Page needed a signal the page author does not control. A page's own words are written by its author. The links pointing to it are written by other people. That is the page's company.

The argument

THE PHILOSOPHY

Their argument came from how academics judge papers. A paper matters if it is cited a lot. But not every citation is equal. A citation from an important paper should count for more. Page and Brin carried this over to links^[^techreport]. A link from the Yahoo home page *“may be just one link but it is a very important one”*^[^techreport].

So the saying, for pages, becomes:

| A page is important if important pages link to it.

Read it again. The definition uses itself:

```
how important is page A?  
  needs: how important are the pages that link to A?  (say B and C)  
    needs: how important are the pages that link to B and C?  
      needs: how important are the pages that link to those?  
        needs: ...  and somewhere down this chain, page A again
```

It looks circular. The authors knew, and they accepted it on purpose¹.

Before you read on: to score page A you need the scores of the pages that link to it, and their scores need A's. How would you get a number at all?

It turns out circular is fine, because we can solve it by repeating a simple step.

Watch it settle

Here is a web of four pages. Page 0 links to pages 1 and 2. Page 1 links to page 2. Page 2 links to page 0. Page 3 links to page 2, and nothing links to page 3.

We do not know anyone's importance yet, so everyone starts equal. Then one rule: every page hands its score out along its links, split equally. Here is round one:

```
start:  page 0 = 0.25   page 1 = 0.25   page 2 = 0.25   page 3 = 0.25
```

```
each page hands out its score along its links:
```

```
page 0 (2 links) ---0.125---> page 1
                        ---0.125---> page 2
page 1 (1 link)  ---0.25-----> page 2
page 2 (1 link)  ---0.25-----> page 0
page 3 (1 link)  ---0.25-----> page 2
                        nobody ----> page 3
```

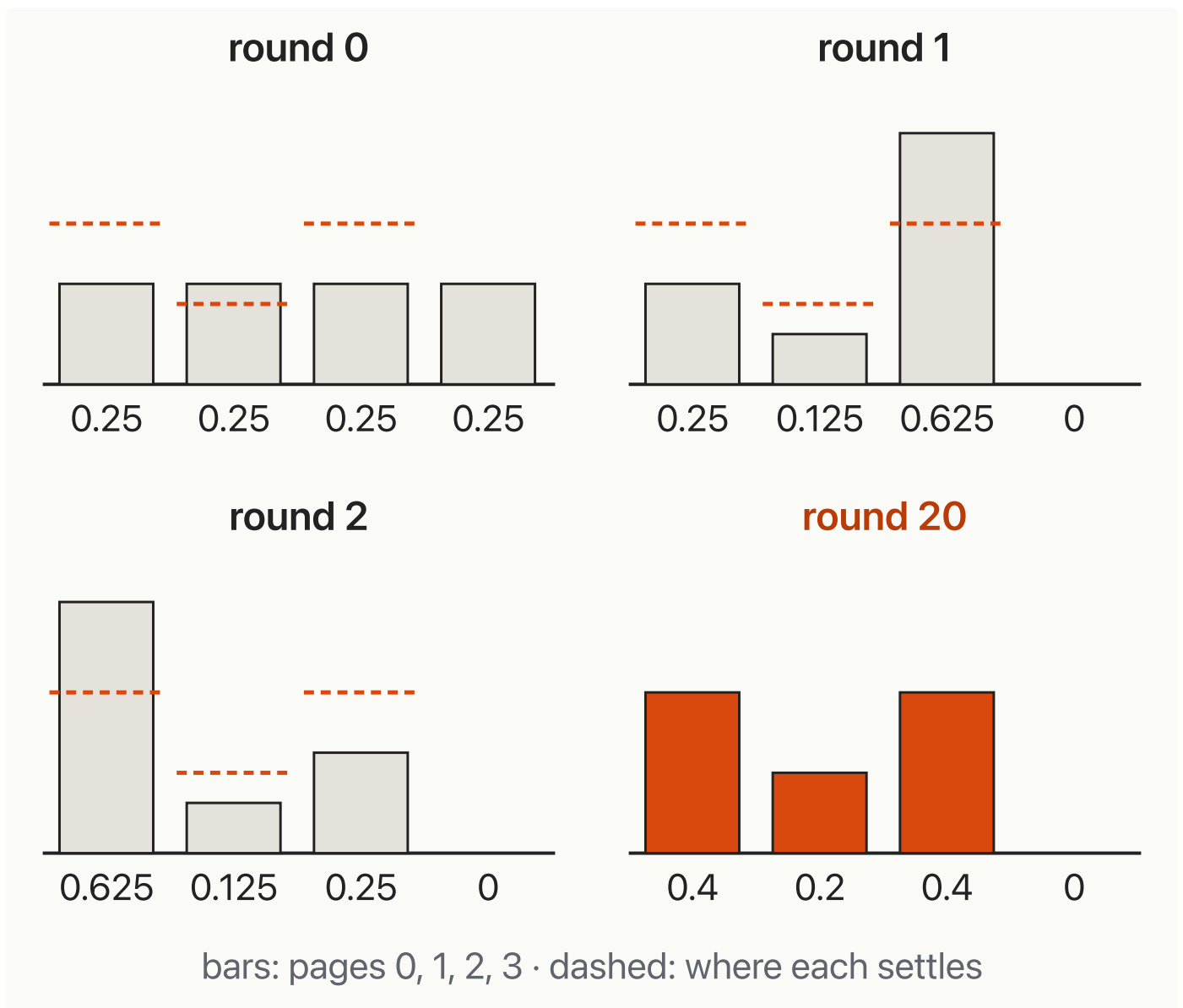
```
after round 1:
```

```
page 0 = 0.25                (from page 2)
page 1 = 0.125               (from page 0)
page 2 = 0.125 + 0.25 + 0.25 (from pages 0, 1 and 3) = 0.625
page 3 = 0                   (nobody links to it)
```

After one round, page 2 has the most. Three pages link to it, so it collected three shares.

Now repeat the round with the new scores. This is where the circular part pays off. Page 2 now has a big score, and it passes all of it to page 0. So page 0 jumps up in round two, from 0.25 to 0.625. It has only one in-link, but that in-link comes from the page that matters most.

Keep going, round after round. The scores swing at first, and then the swings get smaller. After about 20 rounds they stop changing.



Pass the scores along the links, again and again. The swings shrink and the scores settle.

One more rule makes this work on a real web: the random jump. Two pages that link only to each other would soak up score and never let it go. Page and Brin fixed this with a bored surfer². The surfer clicks links at random, but 15% of the time gets bored and types in a new address. In score terms, every round each page passes 85% of its score along its links, and the other 15% is spread evenly over all pages. That is why page 3, which nobody links to, ends with a small score and not zero.

When the scores stop changing, passing them around once more gives back the same scores. That is exactly what the argument asked for. Every page's score now equals what the pages linking to it hand over, plus its share of the random jumps. "Important

pages are linked to by important pages" is true of these numbers. The circular definition had an answer all along. Repeating the step found it.

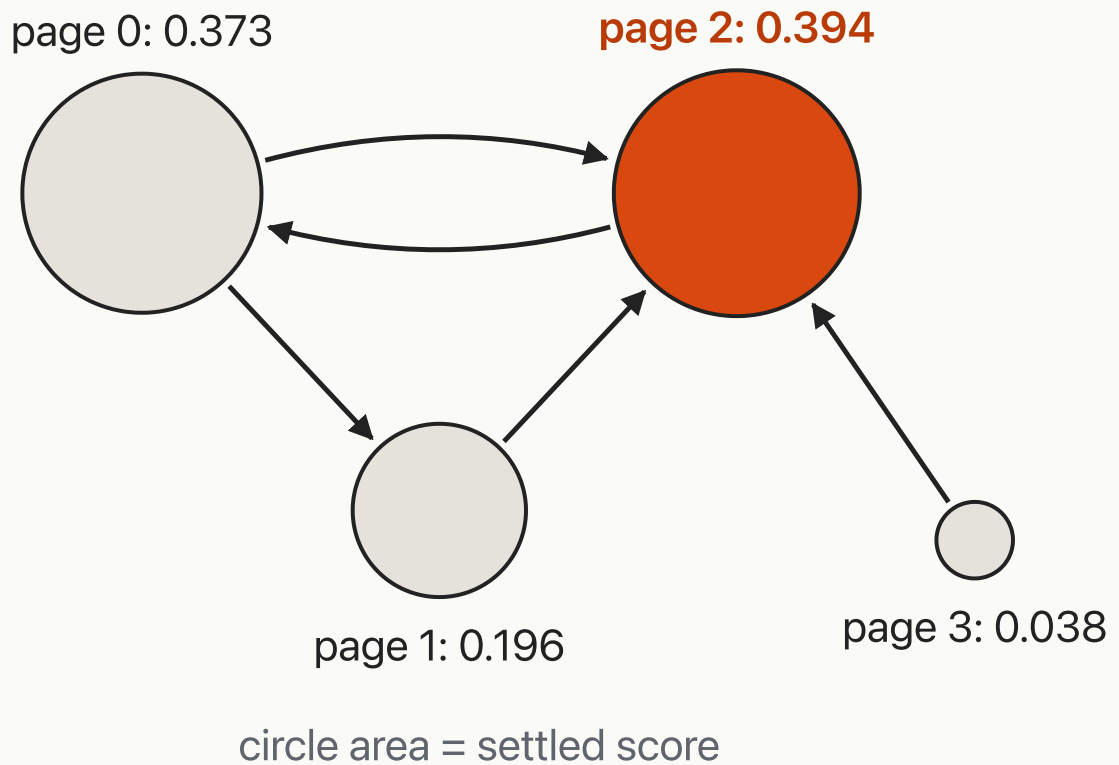
Now the names

Everything so far fits in one line. If r is the list of scores, N the number of pages, $d = 0.85$, and M the table that says how each page splits its score over its links, one round is:

$$r \leftarrow \frac{1-d}{N} + dMr$$

Repeat until r stops changing. That is PageRank². Here is the four-page web in code:

```
1 import numpy as np
2
3 # links[i] = pages that page i links to
4 links = {0: [1, 2], 1: [2], 2: [0], 3: [2]}
5 N, d = 4, 0.85
6
7 M = np.zeros((N, N))
8 for i, outs in links.items():
9     for j in outs:
10         M[j, i] = 1 / len(outs) # i passes its score equally to each page i
11
12 r = np.ones(N) / N
13 for _ in range(50):
14     r = (1 - d) / N + d * M @ r
15
16 print(r.round(3)) # [0.373 0.196 0.394 0.038]
```



The settled scores. Page 0 has one in-link, but it comes from page 2, so it comes second.

Two names come with this. Repeating the same step until nothing changes is called power iteration. The settled scores, the ones that come back unchanged after one more round, are called the principal eigenvector of the table, with the random jump folded in. You do not need either name to understand the idea. You already watched both happen.

THE PHILOSOPHY

So PageRank is the saying, made precise for pages. A page is defined by the company that links to it. We turned that into a table of links, solved it by passing scores around until they settle, and an algorithm fell out.

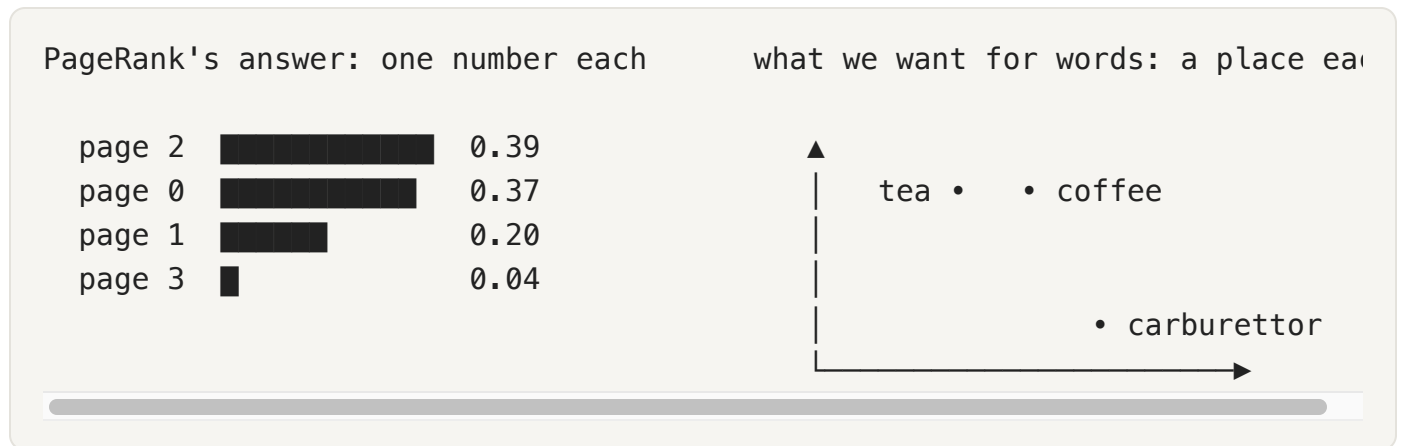
But look at what PageRank hands back: one number per page. That number says how much a page matters. It says nothing about what the page is like.

What if each thing got a description of its company, and not just one score?

Words: predict the company

The problem in 2013

For words we want a different answer: which words are alike?



A single score cannot put tea next to coffee and far from carburettor. A word needs a description of its company: a short list of numbers, a vector, so that words with similar company land in nearby places. (Passing scores around is still the right tool when you do want importance, for example the sentences of a long document. We come back to that in “Swap the tools”, along with the same game played on web pages.)

People had learned word vectors before, but slowly. In 2003, Bengio and co-authors learned them inside a model that predicts the next word, and five passes over 14 million words took more than three weeks on 40 CPUs³. So there were two problems. The models were slow: most of the time went into a hidden layer of extra computation⁴. And the table is huge: a million words give a trillion cells, and almost all of them are zero. The fix below solves both. It drops the hidden layer, and it never builds the table.

Before you read on: you want a short vector for each word, and the table is a trillion cells, almost all of them zero. How would you get the vectors without

building the table?

Mikolov and co-authors at Google wanted good word vectors from far more text than anyone had used⁴. Their intuition was simple. If you only want the vectors, you do not need a model that is good at writing text. A much simpler model *“might not be able to represent the data as precisely”*⁴. But if it only has to get each word’s company roughly right, it should give vectors that are just as good, and read far more text in the same time⁴. So they dropped the hidden layer, and they never built the table. Instead, each word gets a vector with one job: predict its company.

The argument

THE PHILOSOPHY

Linguists had the saying for words long before computers could use it. In 1954 Harris wrote that words with similar meanings occur in similar contexts[^harris]. In 1957 Firth put it as a near copy of our saying: *“you shall know a word by the company it keeps”* [^firth].

Look at the gaps a word can fill:

I drank a cup of ____ this morning	tea fits	coffee fits
she poured hot ____ into the cup	tea fits	coffee fits
the mechanic replaced the ____	carburettor fits	

Tea and coffee fit the same gaps. You learn that they are alike not from a definition, but because they keep the same company. So the saying, for words, becomes:

THE PHILOSOPHY

Two words are alike if they keep the same company.

To turn this into numbers, give every word a vector, and give the vector one job: predict the word's company. Tea's vector has to predict "drank" and "cup". So does coffee's. If two vectors have to do the same job, they should end up in the same place. Let's watch that happen.

Watch tea and coffee meet

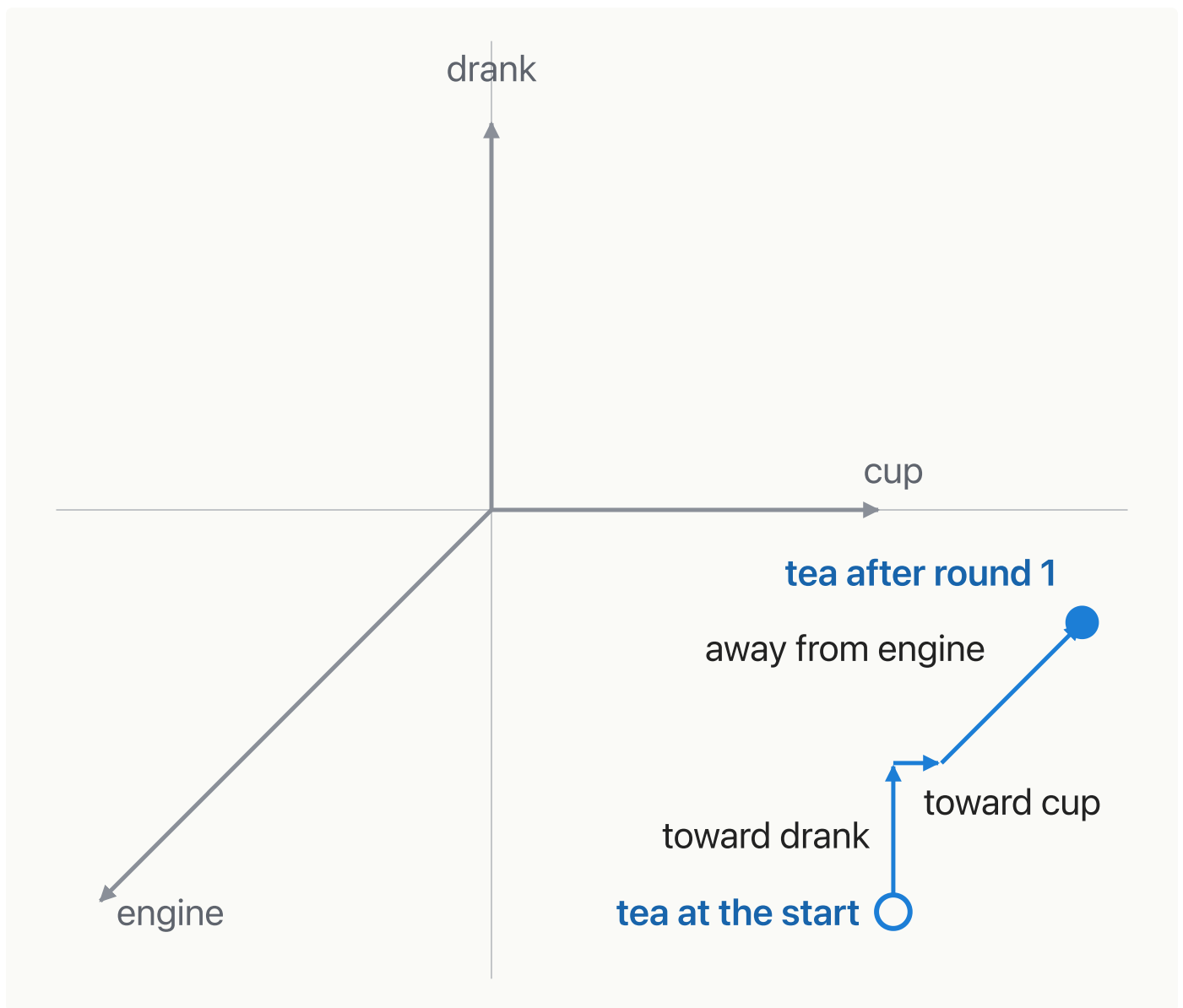
Here is a corpus of three sentences. To keep it tiny, we keep only the words that matter:

I drank tea from a cup	tea's company:	drank, cup
I drank coffee from a cup	coffee's company:	drank, cup
the engine needs a new carburettor	carburettor's company:	engine

Every vector has two numbers, so we can draw it. The company words get fixed vectors: drank = (0, 1), cup = (1, 0), engine = (-1, -1). Tea, coffee and the carburettor start in arbitrary places, far apart: tea = (1, -1), coffee = (-1, 1), carburettor = (1, 1).

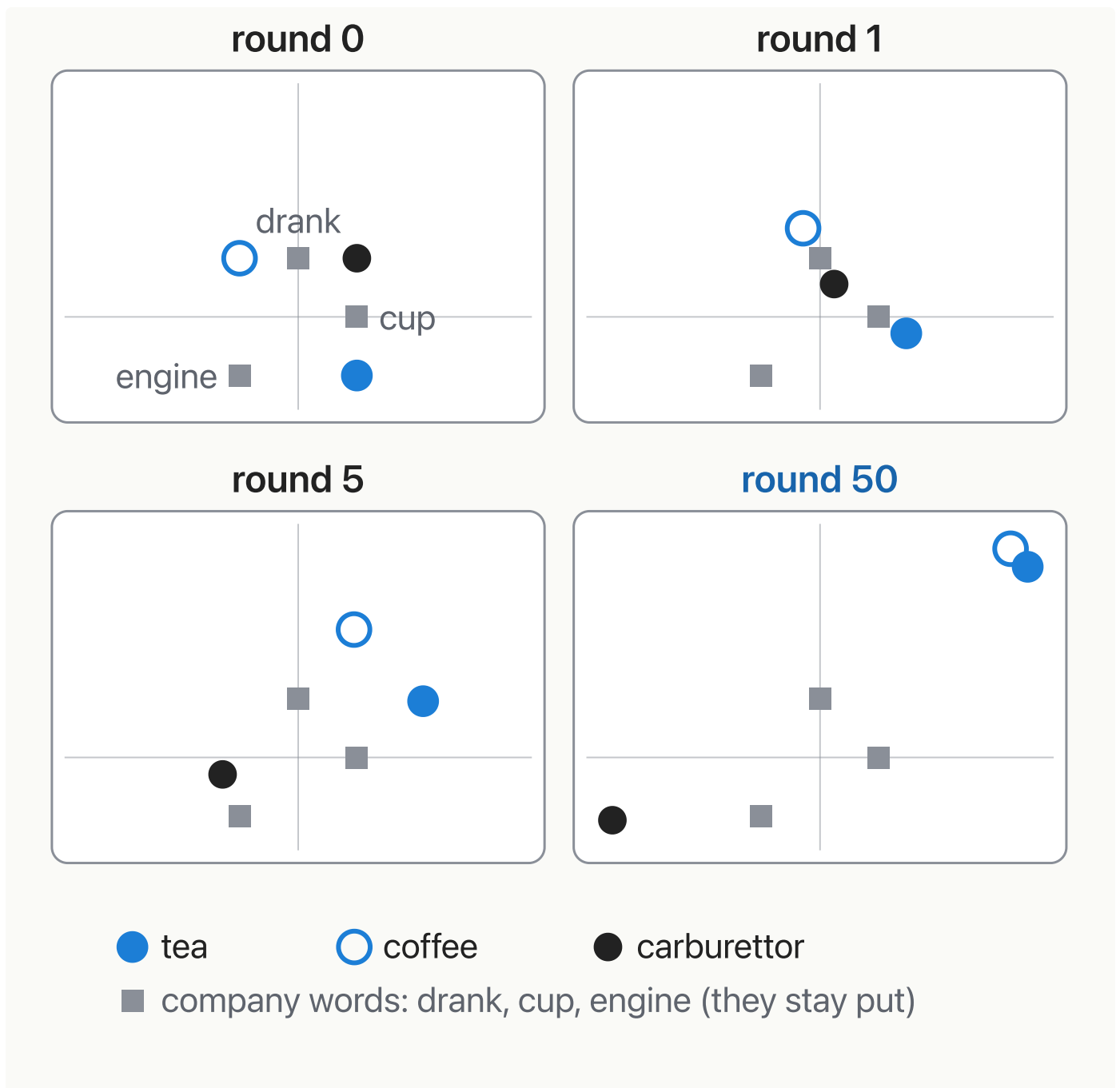
Each round, pull tea toward each word in its company, and push it away from one random word it never sits with. (Why the random word is there comes after the example.)

At the start, tea and drank match badly, so tea moves a long way along drank. Tea already matches cup well, so it moves only a little along cup. A match is how much two arrows point the same way. The worse tea matches a company word, the further it is pulled, and the better it matches the random word, the further it is pushed. And it is pushed away from engine, the random word.



Tea's first round: a long pull along drank, a short pull along cup, a push away from engine.

Now repeat the round with the new vectors, again and again.



Pull each word toward its company, again and again. Tea and coffee meet; the carburettor leaves.

Tea and coffee never appeared in the same sentence. Neither one was ever pulled toward the other. They met because they had to predict the same company. That is the saying, and we watched it happen.

One more rule: the random words. Real word2vec needs them; we kept one, engine, so the toy matches. In our toy the company words stay put. In real word2vec the company vectors move too, and then the game has a trivial solution: put every vector

in the same place, pointing the same way and very long. Every pair matches, the model is always sure, and the vectors say nothing⁵. The random words rule this out. A vector must match its company and also fail to match the words it never sits with.

Now the names

To match two vectors, multiply them number by number and add: a large match is a good match. The match is then squashed to between 0 and 1, which says how sure the game is that the two words belong together.

Everything so far fits in one line. For a word w , a real company word c and k random words $n_1, \dots, n_k$, the game moves the vectors to make this number larger, where σ is the squash:

$$\log \sigma(v_c \cdot v_w) + \sum_{i=1}^k \log \sigma(-v_{n_i} \cdot v_w)$$

The first part grows when the real pair matches, and the second when the random pairs do not. Tea's moves above are exactly the steps that make this number larger. One step touches only $k + 2$ vectors. The authors report that an optimized implementation can train on more than 100 billion words in a day on one machine⁶. Here is the toy in code:

```
import numpy as np
sigma = lambda s: 1 / (1 + np.exp(-s)) # the squash

company = {"drank": np.array([0., 1.]), "cup": np.array([1., 0.]),
           "engine": np.array([-1., -1.])}
vec = {"tea": np.array([1., -1.]), "coffee": np.array([-1., 1.]),
       "carburettor": np.array([1., 1.])}
# each word, its company in its sentence, and one random word
games = [("tea", ["drank", "cup"], "engine"),
         ("coffee", ["drank", "cup"], "engine"),
         ("carburettor", ["engine", "cup"])]

step = 0.5
for _ in range(50):
    for w, comp, rand in games:
        for c in comp:
            vec[w] += step * (1 - sigma(vec[w] @ company[c])) * company[c] #
```

```

1 vec[w] -= step * sigma(vec[w] @ company[rand]) * company[rand] #
1
1 print(np.linalg.norm(vec["tea"] - vec["coffee"]).round(2)) # 0.42

```

A few names come with this. The vector for each word is called an embedding. The guessing game with random words is called negative sampling. Moving the vectors a little after each pair is stochastic gradient descent. That is the algorithm word2vec⁷⁸.

THE PHILOSOPHY

So word2vec is the saying, made precise for words. A word is defined by the words around it. We turned that into a game of predicting the company, and an algorithm fell out.

But look at what word2vec hands back: one vector per word, the same in every

sentences, with engine as the random word:

```

I sat on the river bank      bank's company: river = (1, 0)
I took a bank loan          bank's company: loan  = (0, 1)

after 50 rounds:    bank matches river and loan almost exactly equally

```

Bank ends up almost exactly between the two. In “I sat on the river bank”, half of that vector is about money. word2vec has no way to choose, because the vector was settled before this sentence was read.

One vector has to serve both banks, and the sentence that says which bank is right in front of you. What does the sentence know that the vector does not?

Different company, different mask

You wear masks

THE PHILOSOPHY

The saying leaves something out. You are different in different company. You talk one way to your parents and another way to your friends. You wear multiple masks, and the company you are in decides which one.

This is old for words too. In 1935, Firth argued that a word's full meaning always depends on its context, and that you cannot study meaning apart from context^[^firth35]. The same word means different things in different sentences, because it keeps different company there.

I sat on the river bank

sat river

 \ /
 bank

|
[mask: the edge of a river]

I took a bank loan

took loan

 \ /
 bank

|
[mask: money]

same word, different company, different mask

THE PHILOSOPHY

So the saying, for a word in a sentence, becomes:

| A word wears a different mask in different company.

One face for all company

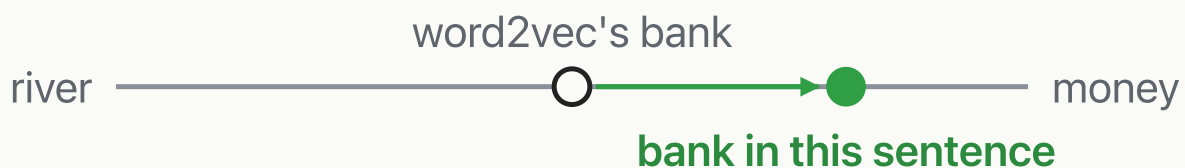
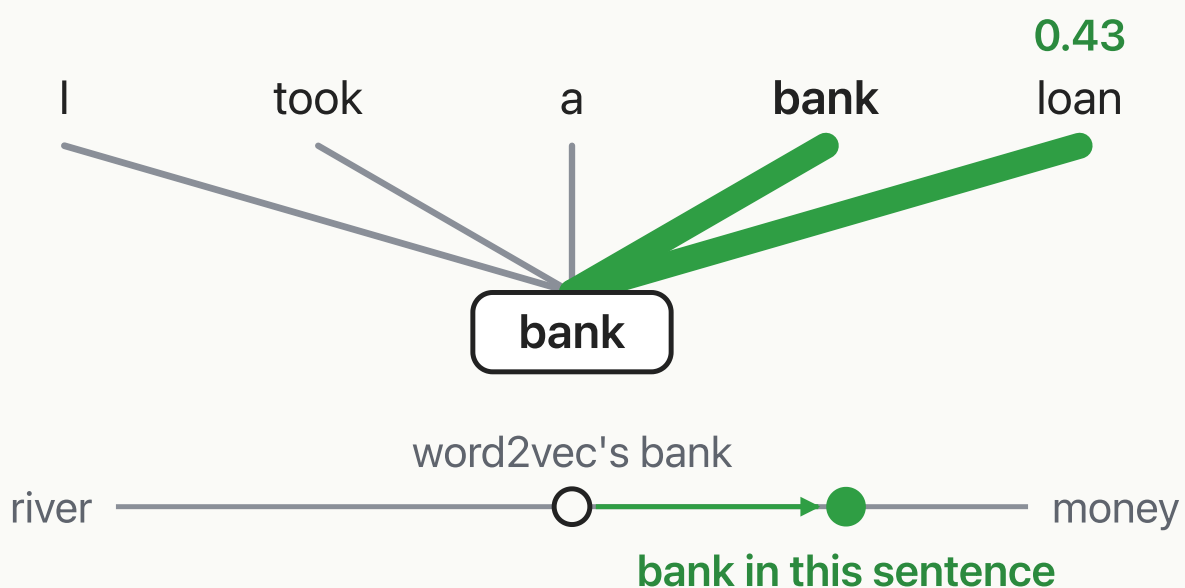
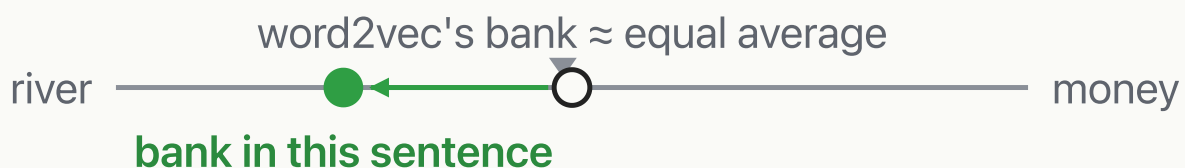
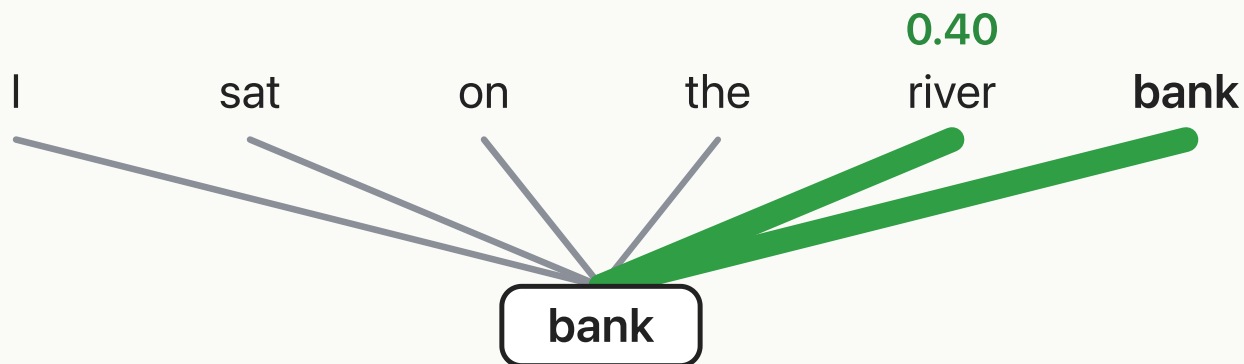
word2vec took the first half of the saying and dropped the second. It learns one vector per word from all of the word's company, in every sentence at once. Then it uses that one vector everywhere. We saw what this does to bank: it ended halfway between the two. One face for all company.

Researchers had already seen this problem in earlier one-vector models, and they named it in technical terms. In 2010, Reisinger and Mooney wrote that one vector per

The simplest version is an equal average of every word in the sentence, bank included. It lands almost exactly where word2vec put bank: halfway (the grey mark in the picture). "River" counts $1/6$, exactly as much as "on". The one word that settles the mask is outweighed by the words that do not.

So weigh each neighbour by how well it matches what bank is looking for. That is a vector too. Bank carries one more vector, which says what it looks for in its company. A word's own vector says what the word is. The look-for vector says what it needs from its neighbours, and the two need not be the same. In this toy we chose it to reward words that lean far toward the river or toward money, so it barely matches the short words.

Here is what that does in both sentences. Each line joins bank to a word in the sentence, and its thickness is that word's weight. Under each sentence, bank's new vector sits on a line that runs from the river end to the money end:



Same word, looking for the same thing. The company decides which lines are thick, and so which way bank moves.

What bank looks for did not change between the two sentences. The company did, so the weights did, so the mask did. "River" got 0.40 of the say, not 1/6. Bank's own line is thick too, because bank matches what it looks for as well as river does. The short words still count a little, but they can no longer outweigh the word that matters.

Every sentence builds its own mask, so we never choose how many masks bank has or which one to wear.

One more rule: where does the look-for vector come from? It is learned, like word2vec's vectors. After each example it is moved, so the weights it picks help the model with the prediction it is trained on, for example translating the sentence into another language, the way word2vec's vectors were moved to predict the company. And every word has one, not only bank.

Now the names

There are three steps. Match each word against what bank looks for, with the same multiply-and-add word2vec uses. Turn the matches into weights: raise e to each match, so every weight is positive, then divide by the total, so the weights add up to one. The new bank is the weighted average.

This is attention. The look-for vector is called the query. The step that turns matches into positive weights that add up to one is called softmax. For a word with query q and words $x_1, \dots, x_n$ in the sentence:

$$a_j = \frac{e^{q \cdot x_j}}{\sum_{k=1}^n e^{q \cdot x_k}}, \quad \text{new vector} = \sum_{j=1}^n a_j x_j$$

Here are both bank sentences in code:

```

1 import numpy as np
2
3 vec = {"I": [0.1, 0.5], "sat": [0.0, 0.6], "on": [0.1, 0.5], "the": [0.0, 0.5]
4       "river": [2.0, 0.0], "bank": [1.0, 1.0],
5       "took": [0.1, 0.4], "a": [0.0, 0.5], "loan": [0.0, 2.0]}
6 q = np.array([1.5, 1.5]) # what bank looks for: the query
7
8 for sentence in ["I sat on the river bank", "I took a bank loan"]:
9     X = np.array([vec[w] for w in sentence.split()])
10    matches = X @ q
11    weights = np.exp(matches) / np.exp(matches).sum() # softmax
12    print(sentence, "->", (weights @ X).round(2))
13 # I sat on the river bank -> [1.22 0.5 ]
14 # I took a bank loan -> [0.44 1.35]

```

In this code, river's match is 3.00 and a short word's is 0.90, about 3 times smaller. After the $e^{\cdot}$ step, river's weight is about 8 times larger. That is why the good matches pull ahead.

A softmax-weighted average like this was in use by 2014¹¹. In 2017, Vaswani and co-authors built a whole model out of it, the Transformer¹². In the Transformer, every word weighs every other word in the same sentence, all at once, each with its own query. It adds a few details, but the core is the formula above. The same word gets a different vector in each sentence: a different mask in each company.

Bahdanau and co-authors in 2014 were fixing a bottleneck. The basic encoder-decoder they started from had to squeeze a whole sentence into one fixed-length vector, and “the use of a fixed-length vector is a bottleneck”¹¹. Vaswani and co-authors in 2017 wanted a model that trains in parallel: computing one word at a time “precludes parallelization within training examples”¹².

When I first thought through this chain, I got as far as asking: what if word2vec, instead of one fixed vector learned from all of a word's company, had a weighted average, or a function in there? I was pretty sure extensions like this existed. That is attention, and I missed it at the time. But it shows how the blocks lead you there.

So attention is still the saying, with its second half put back. A word is defined by its company, and in different company it wears a different mask. Attention builds that mask from the company at hand.

Two kinds of company, three tools. What if we swap them, and try every tool on every kind of company?

Swap the tools

Look at what we have. The saying was the same each time. What changed was the table and the tool. For pages, the company was links, and the tool was “pass it around until it settles”. For words, the company was nearby words, and the tool was “predict the company”. For attention, the company was the same nearby words, and the tool was “weigh the company”.

If the saying is the same, the tools should work on any table. They do. Each filled cell below is a real paper:

	Pass it around until it settles	Predict the company	Weigh the company
Pages and graphs	PageRank, 1998 ²	DeepWalk, 2014 ¹³ ; node2vec, 2016 ¹⁴	Graph attention networks, 2018 ¹⁵
Words and sentences	TextRank, 2004 ¹⁶	word2vec, 2013 ⁴	Transformer, 2017 ¹²
People	Who to Follow at Twitter, 2013 ¹⁷	DeepWalk, 2014, tested on social networks ¹³	GraphRec, 2019, weights learned from the two people’s vectors ¹⁸

Take the first swap: PageRank’s company, word2vec’s tool. PageRank gives a page one number, how important it is. But we could also ask what a page means, by the

company it keeps. Start at a page and click links at random, like the PageRank surfer. Write down the pages you pass, in order. That list is a sentence, and the pages are its words. Run word2vec on many such walks, and every page gets a vector. Pages that keep turning up in the same walks get similar vectors, because they keep the same company. That is DeepWalk, from 2014¹³.

```
a random walk:    page 7 → page 2 → page 0 → page 2 → page 5
read it as:       "page7 page2 page0 page2 page5"      ← a sentence
word2vec:         pages that share walks get nearby vectors
```

The second swap goes the other way: a new kind of company, PageRank's tool. Make the sentences of a long document the things, link two sentences by how much they share, and pass scores around until they settle. The sentences that end on top are the ones the rest of the document keeps pointing back to. That is TextRank, from 2004¹⁶.

Graph attention networks are the third kind. They let each node weigh its neighbours by a learned score, instead of splitting equally as PageRank does¹⁵.

The bottom row is people. Who to Follow, at Twitter, starts from you, walks the follow graph, and suggests the people you keep landing on¹⁷. GraphRec makes your vector a mix of your friends' vectors, and learns how much each friend counts¹⁸.

THE PHILOSOPHY

The saying gives the table. The tool is a separate choice.

| What does each tool say about a person?

People: the same saying, a new table

So far, a page's company has been the pages that link to it, and a word's company has been the words around it. Let's try the saying on people, using the office from the TV

show The Office. What company does a person keep? At work, it is mostly the people they talk to.

Every message Kevin sends Oscar is a small vote. When Kevin needs something, Oscar is one of the people he turns to. Suppose we write down, for everyone in the office, who they message and how often. We get a table that looks exactly like the link table for the web. The rows are people and the columns are people. Each cell says how much of one person's attention goes to another.

Who matters?

With pages, we asked: if a reader clicks links at random, which page do they keep landing on? Here is the same question for an office.

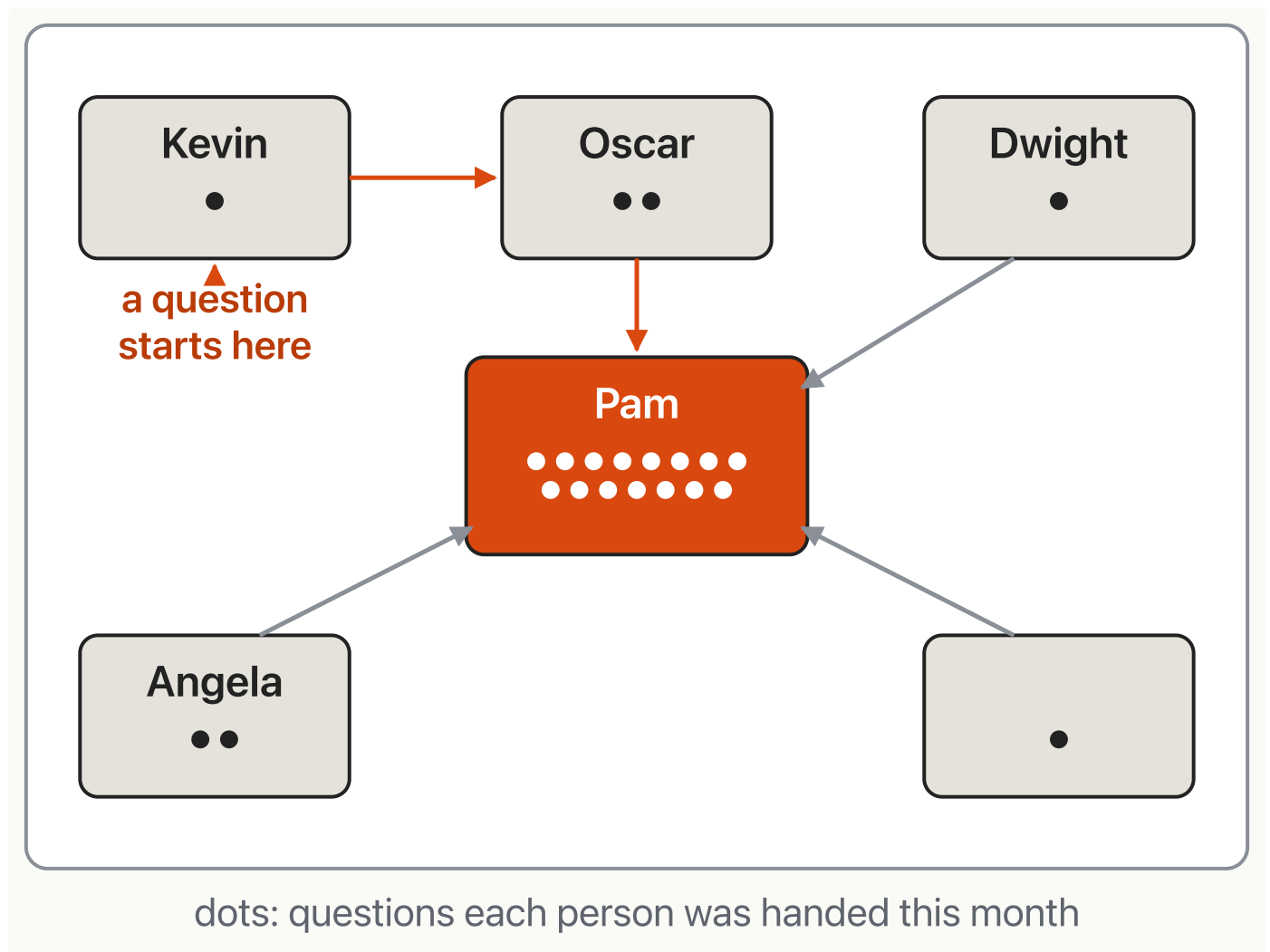
Take a question nobody can answer alone, such as where the expense form lives. Kevin has it first. He doesn't know, so he asks the person he usually asks, Oscar. Oscar doesn't know either, so he asks the person he usually asks. The question moves through the office one conversation at a time. At each step it follows the habits the table records.

Now picture this all month, with hundreds of small questions starting at random desks. Some get answered and stop. Some get dropped, and a new one starts somewhere else. That is the random jump from the PageRank section, and it does the same job here. It stops a question from bouncing forever between two friends who only talk to each other.

Before you read on: at the end of the month, count how many times each person was handed a question. Who ends up on top: the person with the most messages, or someone else? Why?

Pam is far ahead of everyone. She isn't the manager, and nobody gave her this job. She is ahead because many people ask her, and because the people who ask her are asked a lot themselves.

That second part makes this more than counting messages. Being asked by someone who is asked a lot counts for more than being asked by someone nobody talks to. This is the recursive definition we had for pages: important pages are linked to by important pages. The fix is also the same: pass the questions around until the counts stop changing.



A question moves through the office along the habits in the table. Count who is handed questions most often.

There is no new maths here. Swap pages for people and links for messages, and PageRank tells you who the office actually routes through.

Who is like whom?

PageRank gives each person one number. It tells you how central Pam is, but not what kind of person she is. For that we need the tool word2vec used: predict the company.

Recall how word2vec learns: each word's vector has to predict the company around it, and words with the same company end up close. That is why tea and coffee sit close together, even when they never appear in the same sentence.

We can write a person's week as a sentence in the same way. Dwight's week reads: sales calls, client lunch, quota sheet, beet farm. Andy works in the Stamford branch. His week reads: sales calls, client lunch, quota sheet, a cappella practice. Angela's week reads: accounts, expense reports, party-planning committee.

A real week is spread across apps. The calendar says what a person did, the chat app says who they talked to, and their messages and documents say how they do it. All of it can go into the same sentence.

Now hide the name and ask the model who fits: "___ spent Tuesday on sales calls, then updated the quota sheet." The model is right whether it answers Dwight or Andy. To be right both times, it has to give the two of them nearby vectors.

Notice what did not happen. Dwight and Andy have never messaged each other. PageRank would never connect them, because no chain of conversations joins them. word2vec connects them anyway, because they keep the same kind of company. That is the difference between the two tools. The first follows the links that exist. The second finds people who could stand in for each other.

Who is right for this job?

One vector per person is still a compromise. Dwight's vector has to describe the salesman and the beet farmer at once. So for any one question, it is a little wrong. Dwight is "bank" again: one face for all company, when he wears a different mask in each.

Ask the office "who should close this client?", then ask "who should plan the office party?". Dwight has one vector for both questions. Which part of Dwight should answer each one, and who decides? You have already met the tool.

Try it on something new

Here is the test of whether the building blocks are yours. Take songs.

What company does a song keep? The songs next to it on people's playlists. Write that as a table: songs in the rows, neighbouring songs in the columns. Now pick the tool by the question. "Which song should be on everyone's party list?" is a question about importance, so pass it around until it settles. "What sounds like this song?" is a question about similarity, so predict the company: treat each playlist as a sentence and each song as a word. "What should play next, for this listener, right now?" depends on the songs this listener just played, so weigh the company.

THE PHILOSOPHY

You are defined by the company you keep. Decide what the company is. Write it as a table. Pick the tool that answers your question. The algorithm falls out.

Further reading, intuition first

- Brin and Page, [The Anatomy of a Large-Scale Hypertextual Web Search Engine](#) (1998). Section 2.1 gives the random surfer and the citation argument in the authors' own words, in about a page.
- Jay Alammar, [The Illustrated Word2vec](#). The guessing game, drawn step by step, with no maths needed.
- Chris Olah and Shan Carter, [Attention and Augmented Recurrent Neural Networks](#) (Distill, 2016). Interactive diagrams that show which words attention weights, step by step.
- Jay Alammar, [The Illustrated Transformer](#). How every word weighs every other word in the sentence, one picture at a time.
- 3Blue1Brown, [Attention in transformers, step-by-step](#). A video that builds the weights from the matches before it shows the formula.
- Perozzi, Al-Rfou and Skiena, [DeepWalk](#) (2014). The swap from the table above, in a short paper: random walks as sentences.

-
1. L. Page, S. Brin, R. Motwani and T. Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford technical report, 1998. ↩ ↩
 2. S. Brin and L. Page, *The Anatomy of a Large-Scale Hypertextual Web Search Engine*, WWW7, 1998. ↩ ↩ ↩ ↩ ↩
 3. Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, *A Neural Probabilistic Language Model*, JMLR 2003. ↩
 4. T. Mikolov, K. Chen, G. Corrado and J. Dean, *Efficient Estimation of Word Representations in Vector Space*, 2013. ↩ ↩ ↩ ↩ ↩
 5. Y. Goldberg and O. Levy, *word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method*, arXiv 1402.3722, 2014. Section 2.2 points out this trivial solution and why the random pairs are needed. ↩
 6. T. Mikolov et al., *Distributed Representations of Words and Phrases and their Compositionality*, NeurIPS 2013. ↩
 7. O. Levy and Y. Goldberg, *Neural Word Embedding as Implicit Matrix Factorization*, NeurIPS 2014. A year after word2vec, this paper showed that its guessing game is, in effect, compressing the same word-by-company table we decided not to build. The tool changed. The saying and the table stayed the same. For the curious: the table it factorizes is the word-by-context table rescaled by PMI (how much more often two words appear together than chance), shifted by the log of the number of random words. ↩
 8. Two differences from the toy matter here. Every word has two vectors, one for when it is the word and one for when it is the company, and both move. And the company is every word within a few positions, with nearer words counting a little more¹⁹. ↩
 9. J. Reisinger and R. J. Mooney, *Multi-Prototype Vector-Space Models of Word Meaning*, NAACL-HLT 2010. ↩ ↩
 10. E. H. Huang, R. Socher, C. D. Manning and A. Y. Ng, *Improving Word Representations via Global Context and Multiple Word Prototypes*, ACL 2012. ↩ ↩
 11. D. Bahdanau, K. Cho and Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, 2014. ↩ ↩
 12. A. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017. ↩ ↩ ↩

13. B. Perozzi, R. Al-Rfou and S. Skiena, *DeepWalk: Online Learning of Social Representations*, KDD 2014. [↩](#) [↩](#) [↩](#)
14. A. Grover and J. Leskovec, *node2vec: Scalable Feature Learning for Networks*, KDD 2016. [↩](#)
15. P. Veličković et al., *Graph Attention Networks*, ICLR 2018. [↩](#) [↩](#)
16. R. Mihalcea and P. Tarau, *TextRank: Bringing Order into Texts*, EMNLP 2004. [↩](#) [↩](#)
17. P. Gupta, A. Goel, J. Lin, A. Sharma, D. Wang and R. Zadeh, *WTF: The Who to Follow Service at Twitter*, WWW 2013. [↩](#) [↩](#)
18. W. Fan, Y. Ma, Q. Li, Y. He, E. Zhao, J. Tang and D. Yin, *Graph Neural Networks for Social Recommendation*, WWW 2019. [↩](#) [↩](#)
19. O. Levy, Y. Goldberg and I. Dagan, *Improving Distributional Similarity with Lessons Learned from Word Embeddings*, TACL 2015. Discusses word2vec's dynamic window, which gives nearer words more weight. [↩](#)

Search

Recommendations

Pagerank

Word2vec

Attention

Machine-Learning

NEXT »

Eyes on the prize: pick one metric and
make people look at it

